

Demo Day Countdown

Wiring the Cordwell RAG Stack Before Friday

Week 7, Module 01 discussion exercise 2 of 2. Teams of 3. Target 35 to 45 minutes.

The Situation

Riley is back from vacation: tan, rested, and dangerously confident. While you were fixing the review-summarizer, Riley volunteered your team for something bigger. This Friday at 10 AM, the VP of Engineering gets a live demo of Cordwell's first multi-service ML application: the four-service RAG app from the slides. An api-gateway, an embedder, a Pinecone Local vector store, and an llm service, with the actual model running on the host through Ollama or LM Studio.

Riley has already drafted the Compose file ('it mostly works, it just gets moody on startup'). You have one working session to make this stack demo-proof. Three rounds: fix the wiring between containers and the host model, tear down Riley's Compose draft, then run disaster drills so nothing surprises you in front of the VP. All services and data are synthetic.

How This Works

- Teams of 3.
- Slides are open book. Every answer is on slides 19 to 38.
- Each round ends with a written deliverable. Bring one decision per round to the debrief.

Round 1: Sound Check (9 minutes)

Before you can trust the whole stack, you must trust the model wiring. Three quick puzzles. Aim for about three minutes each.

Puzzle 1: The curl that works in one window and not the other

In a terminal on the Mac, this works:

```
$ curl http://localhost:11434/api/tags      # from the Mac terminal
{"models": [ ... ]}                       # Ollama answers
```

Riley then opens a shell inside the running api container and tries the same thing:

```
$ docker compose exec api bash
root@a1b2c3:/app# curl http://localhost:11434/api/tags
curl: (7) Failed to connect to localhost port 11434: Connection refused
```

Your task: explain the failure in one sentence from the container's point of view, write the corrected URL, and answer this follow-up: why does Docker give that hostname a name instead of an IP address?

Puzzle 2: The Thursday night backend swap

Thursday, 6 PM. Riley's Ollama install starts acting up, and the team decides to demo against LM Studio instead. Riley panics: 'Do we have to rebuild the images tonight?'

Your task: list exactly what changes and exactly what does not. Be specific: name the environment variables and values that change, and say what happens to the image, the Dockerfile, and the application code. One slide in the deck answers this almost word for word.

Puzzle 3: The 1.9 GB image and the sorting hat

The llm service image comes in at 1.9 GB and the security review flags 'compilers present in production image.' The fix is a multi-stage build. Sort each item below into one of three buckets: **Builder stage only**, **Final shipped image**, or **Nowhere in the shipped image**. One item legitimately belongs in two buckets; find it and say why.

Item	Bucket (and why, briefly)
The build toolchain (compilers, headers)	
pip's download cache	
The /venv directory with installed packages	
app/ source code	
requirements.txt and the pip install step	
The python:3.13-slim base image	

Deliverable: corrected URL plus one sentence for Puzzle 1, the change list for Puzzle 2, and the completed sorting table for Puzzle 3.

Round 2: The Compose File Riley Drafted (14 minutes)

Here is Riley's draft for the four-service RAG stack. Riley's note: 'Works on the third or fourth try usually. Startup is moody. Also the security folks want a word?'

```
services:
  api:
    build: ./api-gateway
    ports:
      - "5000:5000"
    environment:
      - PINECONE_API_KEY=pc-live-8f3a19c2
      - PINECONE_HOST=http://localhost:5080
      - EMBEDDER_URL=http://localhost:8000
      - LLM_URL=http://llm:8000
    depends_on:
      - pinecone
      - embedder
      - llm

  pinecone:
    image: ghcr.io/pinecone-io/pinecone-local:latest
    platform: linux/amd64
    ports:
      - "5080:5080"
    environment:
      PORT: 5080
      PINECONE_HOST: pinecone.local
    networks:
      default:
        aliases:
          - pinecone.local
```

```

embedder:
  build: ./embedding-service
  ports:
    - "8000:8000"
  environment:
    - MODEL_NAME=BAAI/bge-small-en-v1.5
    - DEVICE=cpu

llm:
  build: ./llm-service
  ports:
    - "8001:8000"
  environment:
    - BACKEND_MODE=ollama
    - OLLAMA_BASE_URL=http://ollama:11434
    - MODEL=gemma4

```

Your tasks

- Task A: Find every defect. There are at least six distinct ones: some break the demo outright, some merely embarrass you in front of the security team. For each, say which slide convicts it.
- Task B: Rank your top three by 'most likely to ruin Friday' and write the corrected lines for each.
- Task C: The follow-up question: with your fixes applied, which single port is reachable from the host, and how does every other hop in a request travel? Trace it against the request flow slide.

Deliverable: the defect list with slide numbers, the ranked top three with corrected lines, and the one sentence answer to Task C.

Round 3: Dress Rehearsal Disaster Drills (11 minutes)

Three drills, drawn from things that will absolutely happen this week. For each: root cause in one sentence, then the exact fix: the command, the Compose stanza, or the pattern.

Drill A: The four minute silence

Between rehearsals a helpful teammate runs `docker compose down` to 'clean things up.' The next `docker compose up` sits for four minutes while the embedder re-downloads its model. Riley whispers: 'Did down just delete our volume?' Answer Riley's question precisely: what does down remove, what does it keep, and what does the four minute wait tell you about Riley's draft? Write the Compose lines that make this a ten second restart forever.

Drill B: The moody startup

Roughly one cold start in three, the api service dies immediately: connection refused talking to the vector store. Running `docker compose up` again always fixes it. Riley calls it moody. You call it a race. Explain the race in one sentence, then write the pattern that fixes it: which service gets a healthcheck, and what changes under the api service's `depends_on`. Why is `service_started` not enough?

Drill C: The VP's follow-up

Wednesday, the VP's assistant emails: 'Great if the demo could show it handling several users at once.' Riley confidently runs `docker compose up -d --scale api=3` and it fails immediately with a port bind

error. Explain the gotcha in one sentence, name the two patterns from the slides that fix it, and pick which one you would actually use for Friday. Defend the pick.

Deliverable: for each drill, the one sentence root cause plus the exact fix, and for Drill C your justified pick.

Stretch Goal (if you finish early)

The VP narration: rehearse the sixty second walkthrough of a single query's journey through the stack, from the user hitting port 5000 to the answer coming back. Name every hop, say which hops travel on the private Compose network, and identify the one hop that leaves the containers entirely. The request flow slide is your script; the goal is to say it out loud without reading it.

Bring to the Debrief

- Your Puzzle 2 change list: what the backend swap actually touches.
- Your number one 'ruins Friday' defect from Riley's Compose file and its corrected line.
- Your Drill C pick and the one sentence defense.